# La lógica del código de la simulación

Qué hace cada paso, y en qué archivo está

Emanuel Orozco Gallego

11 de noviembre de 2026

**Resumen**

Este informe recorre la simulación en el orden en que ocurre, y señala el archivo y la función donde está cada decisión. Las corridas que produjeron los resultados usan el programa en C `src/c/graphene_mc.c`. Python arma la caja, lanza las copias, vigila que sigan vivas y dibuja las figuras. El potencial de Python, en `src/ripples/potential.py`, es la misma fórmula de Tersoff escrita para leerla y para las pruebas; el que corre durante horas es la versión en C.

## 1. El mapa, en una página

Hay tres capas.

1. **La campaña.** `scripts/pipeline_tabla.py` recorre los cinco tamaños, 240, 960, 2160, 4860 y 8640. Si un tamaño ya tiene el archivo `COMPLETO`, lo salta. Al terminar los cinco llama a `scripts/analizar_tabla.py`, que escribe el contraste y las figuras conjuntas.
2. **Un tamaño.** `scripts/pipeline_n8640.py` decide cuántas copias y cuántos hilos, mide cuánto tarda un paso, lanza un proceso por copia y lo reanuda si se muere. Cada copia es el programa `src/c/graphene_mc`.
3. **Una copia.** Dentro de `run_replica`, en el archivo C, se construye la hoja plana, se propone movimientos, se aceptan o se rechazan, y de vez en cuando se anotan la altura, $G(q)$ y $g(r)$.

| Paso | Dónde | Qué decide |
|---|---|---|
| Tamaños y caja | `lattice.py` | $N = 4pq$, lados $L_x$ y $L_y$, hoja plana |
| Campaña | `pipeline_tabla.py` | El orden 240 → 8640 y el salto de lo ya cerrado |
| Reparto de núcleos | `pipeline_n8640.py` | Hilos y copias para llenar 64 núcleos |
| Energía | `graphene_mc.c` | Tersoff, con tablas en memoria |
| Movimiento | `mc_moves` | Un paso = $N$ intentos; Metrópolis |
| Área | `run_replica` | Cada 5 pasos, dilatar o contraer la caja |
| Medición | `sample_observables` | $h$, $G(q)$ y $g(r)$ |
| Figuras | `generate_plots.py` y `analizar_tabla.py` | Curvas y fotos, después de cerrar |

Los nombres cortos de la tabla están en `lattice.py`, en la carpeta `scripts/` y en `graphene_mc.c`.

## 2. Paso 1. Construir la hoja plana

El paper parte de grafeno perfectamente plano, para no meter una ondulación a mano. El código hace eso poniendo la coordenada $z$ de todos los átomos en cero.

La red es la de la Tabla I. Hay dos enteros, $p$ y $q$, y el número de átomos es $N = 4pq$. El lado largo en $x$ es $p$ veces el vector de la red, y el lado en $y$ es $q$ veces tres enlaces. El enlace

vale 1.42 Å. Esa cuenta está en `paper_box` y `graphene_sheet`, en `src/ripples/lattice.py`, y se repite en C en `make_lattice`. Las dos tienen que coincidir: Python elige $p$ y $q$, y C construye las coordenadas con la misma fórmula.

Los cinco tamaños están en el diccionario `PAPER_SIZES`. $N = 19940$ no está, porque el tope pedido es 8640. Para $N = 960$ la tabla impresa del paper dice un lado de 49.29 Å y la fórmula da 49.19 Å. Se usa la fórmula, la misma que cierra las otras filas. El comentario está junto a `TABLE_I_BOX`.

## 3. Paso 2. Decidir cuántas copias corren a la vez

Una sola copia de la caja grande no llena la máquina, y sesenta y cuatro copias de esa caja no caben en la memoria de forma útil. `pipeline_n8640.py` tiene la tabla `REPARTO`:

| $N$ | hilos por copia | copias |
|---|---|---|
| 240 | 1 | 64 |
| 960 | 8 | 8 |
| 2160 | 16 | 4 |
| 4860 | 32 | 2 |
| 8640 | 32 | 2 |

En todos los casos, hilos por copias da 64, el número de núcleos. La caja chica no admite muchos hilos dentro de una copia: los átomos que se mueven a la vez tienen que estar a más de 9.5 Å, y una caja de 25 Å casi no tiene sitio para eso. Por eso $N = 240$ usa 64 copias de un hilo, y las cajas grandes usan pocas copias con muchos hilos.

`aplicar_reparto` recorta esos números si la máquina tuviera menos núcleos. `lanzar` arranca cada copia con `taskset`, de modo que la copia $k$ ocupa los núcleos que le tocan y no pisa a la vecina. La orden que recibe el programa C lleva $p$, $q$, la temperatura, los pasos de equilibrio, los de promedio, la semilla y las rutas de salida.

## 4. Paso 3. La energía de una configuración

Cada vez que se propone un movimiento hay que saber si la energía sube o baja. Esa energía es Tersoff, parametrizado por Lindsay y Broido para carbono. En C los números están al principio de `graphene_mc.c` (`TER_A`, `TER_B`, y el resto). En Python, la misma fórmula está en la clase `TersoffCarbon` de `src/ripples/potential.py`, para las pruebas y para leerla. No es LCBOPII: no hay canal de conjugación. El informe de la reimplementación explica por qué eso cambia las gráficas. Aquí solo importa dónde se calcula.

Tersoff mira, para cada enlace, la distancia y el ángulo con los otros vecinos. El corte está entre 1.80 y 2.10 Å: más lejos, ese vecino no cuenta. Calcular la fórmula exponencial millones de veces es lo lento. `lut_init` la tabula una vez en memoria: una tabla de la distancia, una del ángulo y una del orden de enlace. Durante la corrida, `pair_term_fast` interpola esas tablas. La energía de toda la hoja es `energy_fast`. La de un solo átomo y sus vecinos, que es lo que cambia en un movimiento, es `local_energy_fast`.

Para no recorrer los $N$ átomos en cada intento, la caja se parte en celdas. `cells_bind` y `neighbors_all` guardan, para cada átomo, solo los que caben en el corte. Mover un átomo actualiza su celda en `move_atom_cell` y rehace sus vecinos en `rebuild_atom_safe`.

## 5. Paso 4. Un paso de Monte Carlo

Esto está en `mc_moves` y `apply_prop`.

Un paso son $N$ intentos. En cada intento se elige un átomo al azar y se le suma un desplazamiento al azar en $x$, en $y$ y en $z$, de tamaño como máximo el paso actual (empieza en 0.04 Å). Las coordenadas $x$ e $y$ se pliegan a la caja: si un átomo sale por un lado, entra por el opuesto. Eso es `wrap`. La altura $z$ no se pliega, porque la hoja no es periódica en la dirección perpendicular.

Se calcula la energía local antes y después. Si baja, el movimiento se queda. Si sube, se acepta solo cuando un número al azar entre 0 y 1 es menor que $e^{-\Delta E/kT}$. Si no, el átomo vuelve a donde estaba, incluida su celda. $T$ entra como $\beta = 1/(k_B T)$ en `run_replica`, con $k_B$ en eV por kelvin, así que la energía y la temperatura están en las mismas unidades.

Varios átomos del mismo paso se evalúan a la vez, en paralelo con OpenMP, pero solo si están a más de 9.5 Å (`SEP_MIN`, función `separated`). A esa distancia no comparten vecinos de Tersoff ni la misma celda, así que cambiar uno no falsea la energía del otro. Si no se encuentra un átomo tan lejos, el intento se hace solo. Por eso la caja de 240 átomos no gana nada con muchos hilos: casi nunca hay dos átomos elegibles a la vez.

## 6. Paso 5. Dejar que la caja respire

Cada cinco pasos, todavía dentro del bucle de `run_replica`, se propone multiplicar $L_x$ y $L_y$ por un mismo factor, y todas las coordenadas $x$ e $y$ por ese factor. La hoja se agranda o se encoge sin cambiar de forma. Se recalcula la energía de toda la caja.

Aceptar o no ya no es solo la energía. Al cambiar el área cambia el volumen del espacio de configuraciones, y el paper lo cuenta con un término $N \ln(A'/A)$. En el código eso es `n * 2 * log(factor)`, porque el área escala con el cuadrado del factor. Si el intento se rechaza, coordenadas y lados vuelven atrás y se reconstruyen celdas y vecinos.

## 7. Paso 6. Equilibrar, promediar y no perder el trabajo

El bucle de `run_replica` corre $4 \times 10^5$ pasos de equilibrio y luego $5 \times 10^5$ de producción. Durante el equilibrio no se acumulan $G(q)$ ni $g(r)$: la hoja todavía está saliendo del plano. Al cruzar el último paso de equilibrio se guarda la altura de ese momento.

En la producción, cada cierto número de pasos se llama a `sample_observables`. El ritmo lo elige `medir`, en el pipeline, según lo que tarde un paso de prueba, para no llenar el disco ni saltarse la estadística. Cada muestra hace tres cosas:

1. Resta la altura media, para que $h$ sea la ondulación y no un desplazamiento de toda la hoja. `height_rms` es la raíz cuadrática media de esas $h$.

2. Para una lista de vectores de onda $(n_x, n_y)$, calcula $G(q)/N = q^2|h_q|^2/N$, con $h_q$ la suma de $h$ por la onda plana. La lista no es todos los vectores posibles: `want_mode` se queda con $q \leq 1.25$ Å$^{-1}$, que es la zona de la ondulación larga y de $G_0$, con las coronas de los dos Bragg (2.95 y 5.11 Å$^{-1}$) y con uno de cada 32 modos hasta $q = 6.05$. Guardarlos todos en la caja de 8640 átomos no aporta a la curva y sí al tiempo.

3. Cuenta parejas de átomos hasta 3.5 Å, en 350 intervalos de 0.01 Å, y las acumula en `rdf`. Al escribir el archivo, `write_outputs` las pasa a $g(r)$ dividiendo por el área del anillo y por la densidad, de modo que a distancia grande $g$ tiende a 1.

Cada 200 pasos se añade una línea a `replica_XX_trace.csv`: paso, energía, altura, lados de la caja y aceptaciones. Eso es la gráfica de equilibrio.

Si el proceso recibe una señal de parada, escribe un checkpoint y sale con código 75. El checkpoint, en `ckpt_write` y `ckpt_read`, guarda posiciones, energía, el paso, el generador de azar y los acumulados. Al volver a lanzar la copia, `run_replica` lee ese archivo y sigue en el paso siguiente, sin repetir el equilibrio. El pipeline, en `vigilar`, mira el latido (`beat_write`): si una

copia no escribe durante diez minutos, la mata y la relanza desde el checkpoint. `elegir_resume` es quien pasa la ruta del checkpoint. Un estado con energía no finita o con una altura por encima de 25 Å se rechaza en `state_ok`: no se guarda un NaN como si fuera una muestra.

## 8. Paso 7. Cerrar un tamaño y comparar

`generate_plots.py` se llama desde `cerrar` cuando todas las copias de un tamaño terminan. Lee los CSV de `results/nN/datos/` y escribe las figuras de esa caja: la foto, $G(q)$, $g(r)$, la traza y la altura por copia.

La curva de $G(q)$ no une cada modo de Fourier. La función `spectrum_curve` toma la mediana dentro de una ventana de $q$. Está en `observables.py`. Unir las estrellas una a una dibuja una sierra que no es la curva del paper.

La recta $G_0$ sale de `G0_harmonic`, esto es $T/(\kappa S_0 q^2)$, con $\kappa = 1.1$ eV. Los ejes de la figura son los de la Fig. 3 del paper, $q$ de 0.05 a 5 Å$^{-1}$ y $G/N$ de $10^{-3}$ a 1, para poder mirarlas juntas. La foto en tres dimensiones amplía el eje $z$ seis veces y lo escribe en el título: sin eso, dos ångströms de altura ocupan en el dibujo lo mismo que el lado de la caja.

`contrastar_paper.py` resume cada tamaño en un texto: altura, energía, pico de $g(r)$ y $G/G_0$. Al cerrar los cinco, `analizar_tabla.py` junta esos resúmenes, dibuja las cinco curvas en un solo eje y escribe el documento de la Tabla I.

## 9. Qué comprueba el propio programa antes de fiarse

`compare_flat`, llamado al arrancar con la prueba interna, construye la hoja plana de 240 átomos, compara la energía rápida con la energía calculada sin tablas, y comprueba que no haya vecinos mal contados. La caja grande tiene la prueba `self_test_large`. Si la tabla y la fórmula exacta se separan más de lo tolerado, el programa no sigue. Las pruebas de Python, en `tests/`, repiten la caja, el enlace de 1.42 Å y, para $N = 8640$, el contraste con el paper.

## 10. El recorrido de un átomo, de principio a fin

Una copia empieza en `main` del programa C, que lee los argumentos y llama a `run_replica`. `make_lattice` pone los átomos en el plano. `lut_init` prepara Tersoff. `ws_bind_geometry` hace las celdas. A partir de ahí, durante novecientos mil pasos, `mc_moves` propone desplazamientos y `apply_prop` los acepta o los deshace. Cada cinco pasos la caja cambia de área. Pasado el equilibrio, `sample_observables` acumula. Al salir, `write_outputs` deja cuatro archivos por copia: el resumen, $G(q)$, $g(r)$ y la foto final. El pipeline los lee, y las figuras salen de ahí. Nada de lo que se ve en una gráfica se calcula en el dibujo: el dibujo solo promedia y recorta lo que esos archivos ya traen.